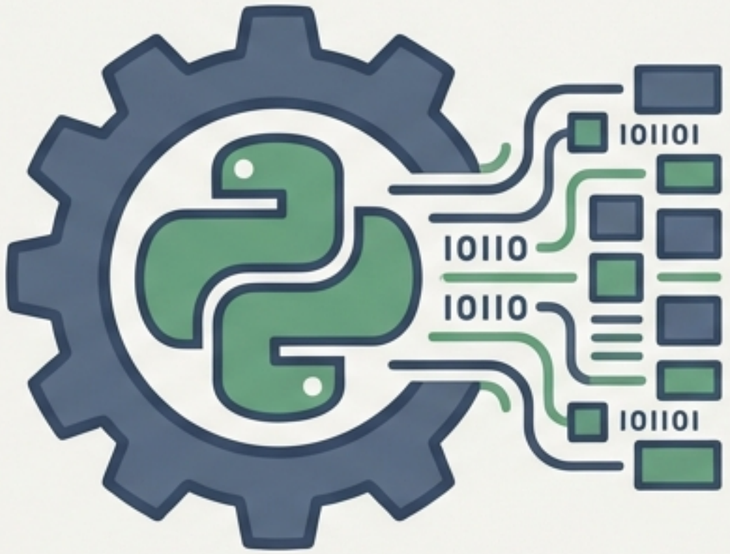
Automated Web Scraper for Academic Data Extraction

Semester Project in Data Pipeline Construction



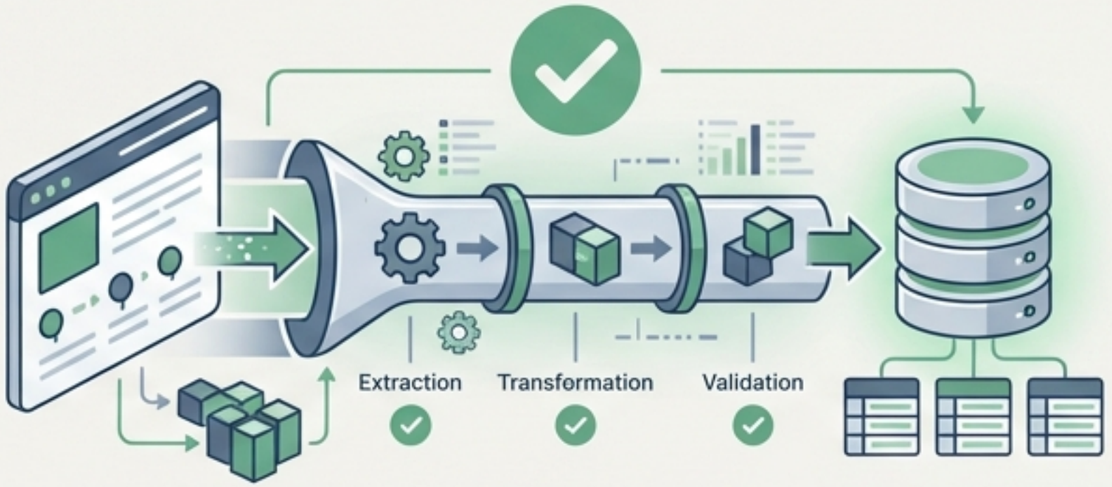
Presented By:
Humna Imran (2023-BS-AI-017)
Ayesha Imran (2023-BS-AI-057)
Mahan Nisar (2023-BS-AI-058)

The Challenge: Manual Data Collection is Untenable

Manually gathering academic information from the Target University Portal is a slow and unreliable process. We identified two primary technical hurdles:

1. **Dynamic Content:** Critical data is fragmented across hundreds of individual pages for courses, fees, and departments, requiring extensive manual navigation.
2. **Inconsistent Formatting:** The website presents data in multiple, incompatible formats. Course lists are in standard `<table>` structures, while fee schedules use complex tables with multi-level, nested headers. Policies are found in unstructured <p> tags, making a single scraping approach impossible.

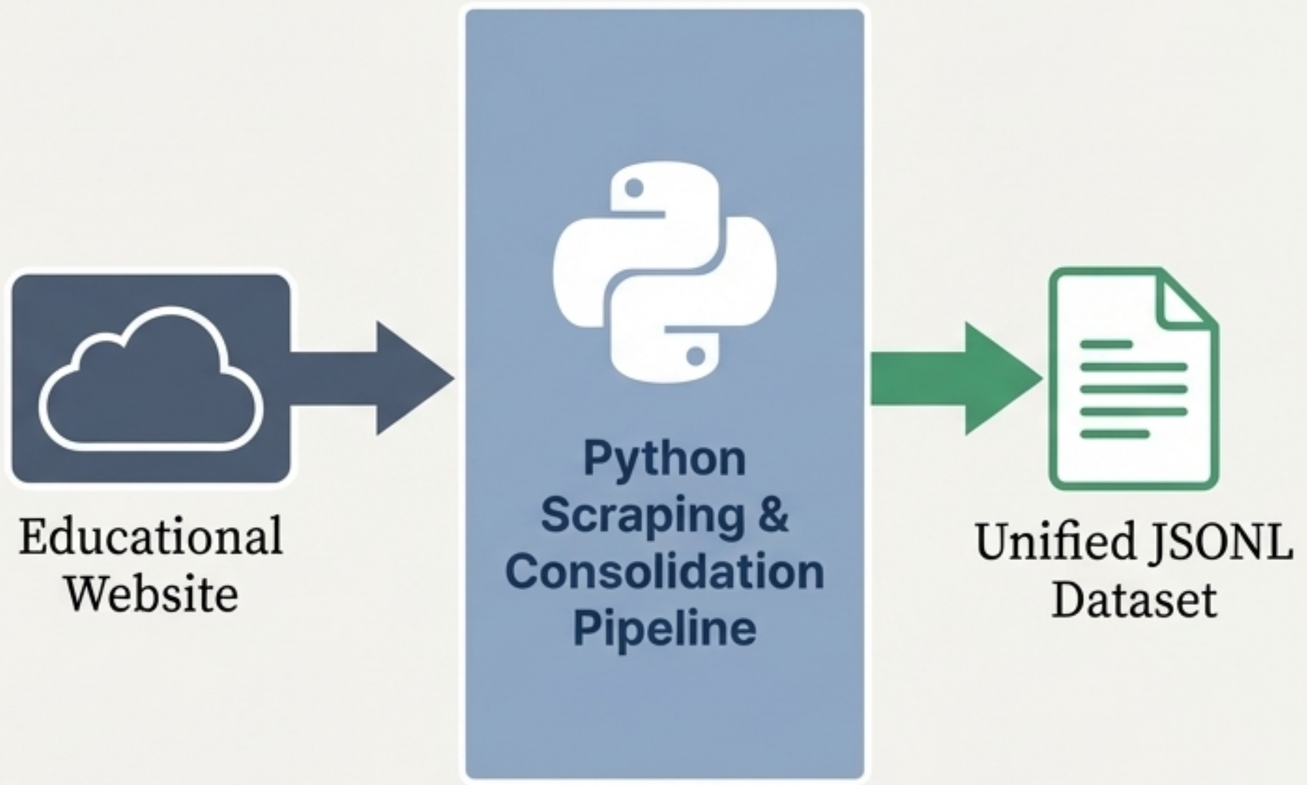
Manual Data Collection **Vs.** **Automated Data Pipeline**



Objective: An End-to-End Automated Pipeline

Our goal was to build a single, automated Python script to overcome the data collection challenges.

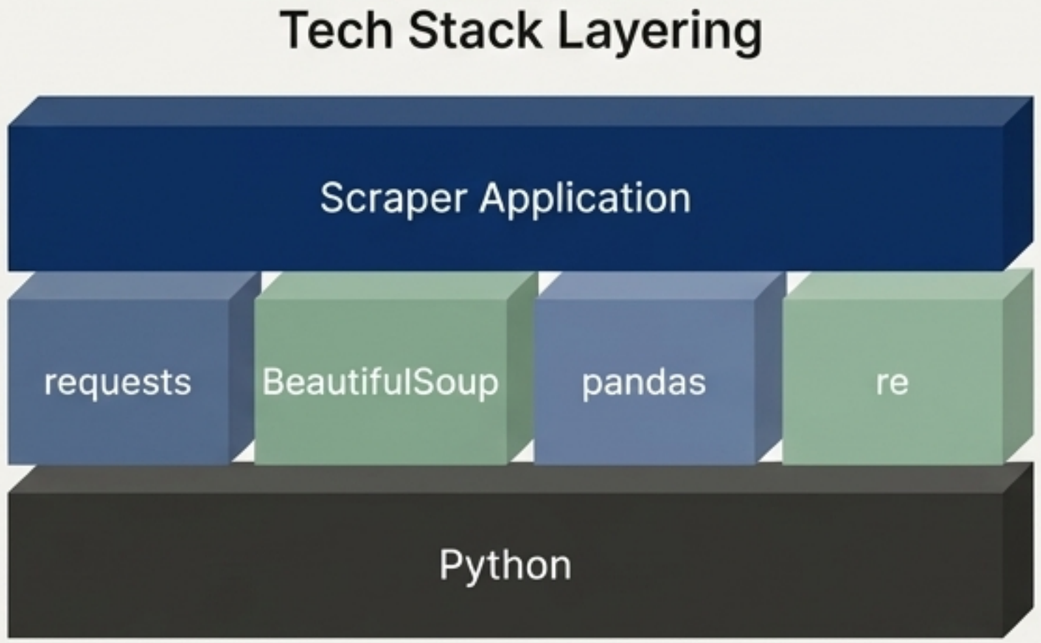
- **Objective:** Systematically navigate the educational website to extract four key data categories: Courses, Fee Structures, Institutional Policies, and Departments.
- Solution: The script processes these varied sources and consolidates them into a single, unified dataset.
- Final Deliverable: A structured, query-ready master_dataset.jsonl file, where every entry is programmatically tagged by its data type (e.g., "course", "fee").



The Technology Stack

We selected a stack of proven Python libraries to build our pipeline:

- **requests:** Used for all HTTP GET requests to fetch the raw HTML content from the target portal's pages.
- **BeautifulSoup:** The core parsing engine. It transforms raw HTML into a navigable Document Object Model (DOM) tree, allowing us to select specific elements like tables and paragraphs.
- **pandas:** Essential for structured data. We leveraged its **read_html** capability to instantly convert HTML tables into DataFrames for easy manipulation and export to CSV.
- **re & json:** The **re** module was critical for data cleaning, using regular expressions to standardize text. The **json** library handled the final serialization of our data into the JSONL format.



Architecture: The Master Pipeline

The script's execution is controlled by the `if __name__ == "__main__":` block, which acts as the central orchestrator for the entire pipeline. This ensures a predictable, sequential workflow.

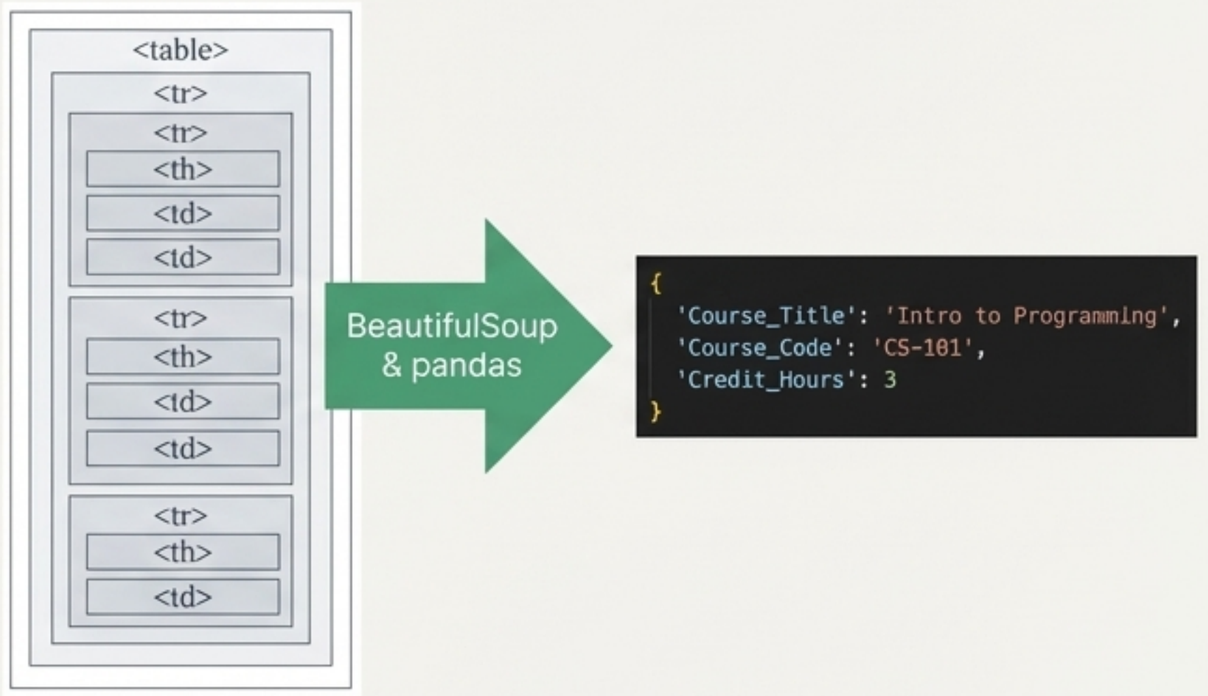
Execution Flow:

- 1. Initiate Course Scraper Module.
- 2. Initiate Fee Scraper Module.
- 3. Initiate Policy Scraper Module.
- 4. Initiate Department Scraper Module.
- 5. Run Final Data Consolidation.

Module 1: Course Scraper Logic

The `scrape_course_page` function is designed to handle well-structured curriculum tables.

- Process: It sends a request to a course URL and uses BeautifulSoup to find all `<table>` tags.
- Validation: To ensure it's processing the correct table, the logic inspects the table headers for specific keywords such as 'Code', 'Title', or 'Credit Hours'.
- Extraction: For each valid table row, the script extracts the text from each cell and maps it into a Python dictionary, pairing keys like "Course_Code" with their corresponding values.



Module 2: Complex Fee Extraction

The fee pages presented a significant challenge with tables using multi-level headers (e.g., a “Faculty of Sciences” header spanning multiple sub-columns). The `scrape_fees_v2` function was built to solve this.

- **Core Logic:** We used `pd.read_html(..., header=[0, 1])`, which directly instructs pandas to parse the first two rows as a hierarchical `MultiIndex`.
- **Header Flattening:** The script then programmatically iterates through the resulting `MultiIndex` tuples (e.g., `('Tuition Fee Per semester', 'Faculty of Sciences')`) and flattens them into a single, clean header like `"Tuition Fee Per semester Faculty of Sciences"`, making the data immediately usable.

Before: Complex Multi-Level Headers

	Faculty of Sciences		
	Tuition Fee Per semester	Enrolment Fee Per semester	Total Fee Per semester
Program A	£10,000	£500	£10,500
Program B	£10,000	£500	£10,500
Program C	£10,000	£500	£10,500

Header Flattening Logic

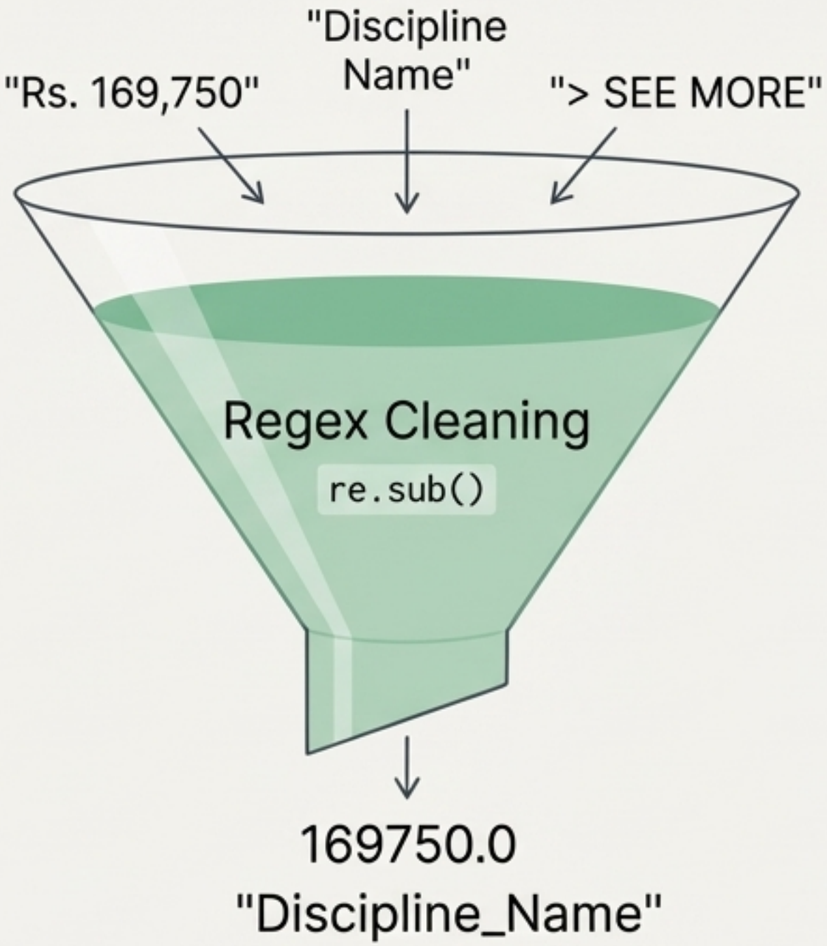
After: Flattened & Clean Data Table

Program	Tuition_Fee_Per_semester_Faculty_of_Sciences	Enrolment_Fee_Per_semester_Faculty_of_Sciences	Total_Fee_Per_semester_Faculty_of_Sciences
Program A	£10,000	£500	£10,500
Program B	£10,000	£500	£10,500
Program C	£10,000	£500	£10,500
Program D	£10,000	£500	£10,500

Data Cleaning Strategy: From Raw Text to Usable Numbers

Scraped data is rarely clean. We implemented dedicated helper functions to ensure data quality and and consistency.

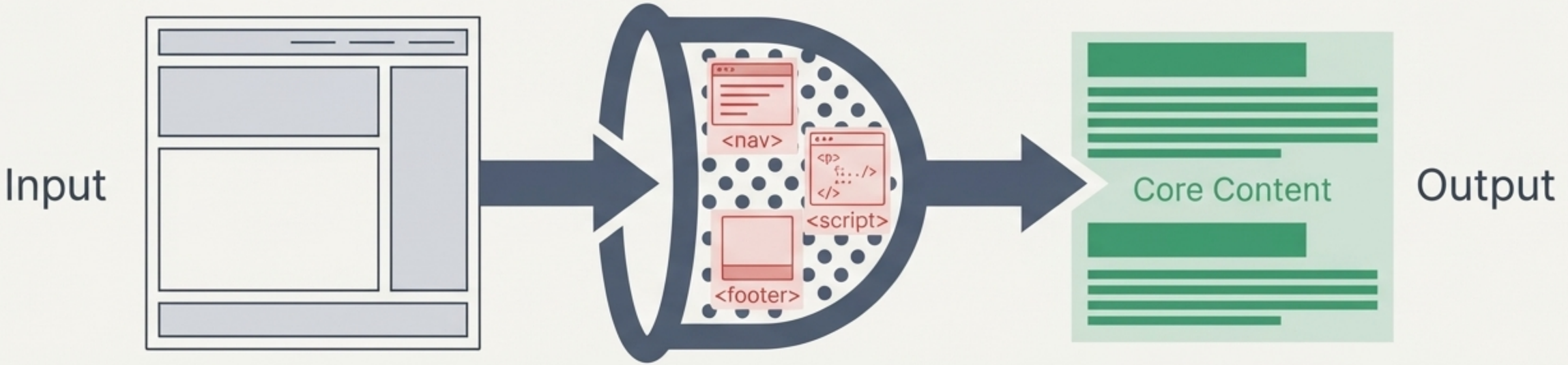
* `clean_currency`: This function uses a regular expression (`re.sub(r'[^\d.]', '', ...)`) to process currency strings. It takes raw text like "Rs. 50,000" or "Rs 25,000", strips all non-numeric characters, and converts the result into a standardized `float` (e.g., `50000.0`).
* `clean_header`: Standardizes column names by removing newline characters and stripping extra whitespace, preventing errors in data processing.



Module 3: Extracting Unstructured Policy Text

Policy pages contain critical information within unstructured paragraphs, not tables. The `scrape_policies_clean` function was designed to isolate this core text.

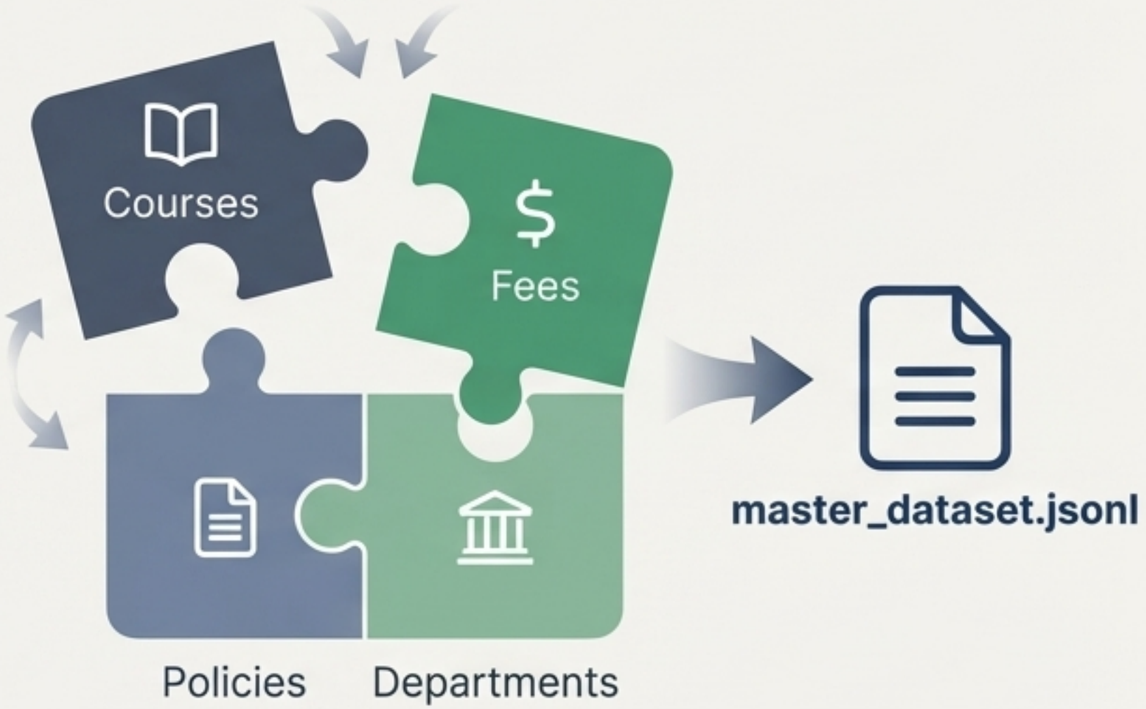
- **Decomposition Strategy:** Before parsing, the script uses Beautiful'Soup's `.decompose()` method to find and programmatically remove all irrelevant HTML sections. This includes navigation bars (<nav>), headers, footers (<footer>), scripts, and sidebars (<aside>).
- **Content Isolation:** This 'de-junking' process leaves only the main article content. The script then iterates through the remaining header tags (<h2>, <h3>) and paragraph tags (<p>) to extract policy titles and their corresponding text.



The Final Step: Data Consolidation

The `run_consolidation` function is the last stage of the pipeline, responsible for creating the master dataset.

- **Process:** It systematically reads the four intermediate CSV files (courses, fees, policies, departments).
- **Transformation:** It iterates through every row of each file, constructs a standardized text summary, and adds a critical "type" key (e.g., "type": "policy"). This tag is essential for identifying the data's origin.
- **Output:** Each processed row is written as a single JSON object to the master_dataset.jsonl file, resulting in a clean, line-delimited JSON format.



Challenges, Robustness, and Optimization

To ensure our pipeline runs reliably and responsibly, we incorporated two key features:

Error Handling

Every network request and file operation is wrapped in a `try...except` block. This prevents the entire script from crashing if a single page fails to load or a table is malformed. The scraper logs the error and continues processing the remaining URLs.

Rate Limiting ("Polite" Scraping)

We added a `time.sleep(1)` command between each HTTP request. This one-second pause prevents our script from overwhelming the educational website's server, minimizing our impact and avoiding potential IP bans.

Conclusion: A Query-Ready Dataset for Future Applications

Summary: Our project successfully delivered an automated, multi-module pipeline that extracts and structures varied data from the Target University Portal.

Final Result: The script produces a clean, consolidated `master_dataset.jsonl` file. This unified dataset is now optimized for downstream applications.

Future Use-Cases: The structured text is ready to be used for:

- Fine-tuning a Large Language Model (LLM).
- Populating a searchable database.
- Performing large-scale data analysis.


```
{"type": "course", "text": "In the BS Computer Sciences program, during Semester 1,
 there is a course titled 'Programming Fundamentals' with code CS-101...",
"source": "..."}
{"type": "fee", "text": "The fee structure for the Doctor of Physical Therapy
program (Faculty: Faculty of Medicine & Allied Health Sciences) is as follows:
Tuition Fee per semester: 178500.0...", "source": "..."}
{"type": "policy", "text": "Regarding Scholarship Policy, specifically the section
'General Introduction': The University offers various scholarships to
'General Introduction': The University offers various scholarships to deserving
students...", "source": "..."}
```